FlashCart Viva & Lab Evaluation Guide

UCS503 Software Engineering • Thapar Institute of Engineering and Technology • Submitted to: Jhonsy Bansal
Pranay Nagpal (1024031127) • Suryansh Jain (1024031145) • Srijan Tanwar (1024031144) • Group 3C82

1 The project in one minute

OPENING PITCH "Good morning ma'am. Our project is **FlashCart**, a unified checkout for shopping malls. Today every showroom has its own billing counter: buy a shirt, earbuds and a coffee and you stand in three queues, pay three times and get three GST bills. With FlashCart the customer signs in with just their **mobile number**, scans the **barcode already on the product tag** with their phone, and every item from every showroom goes into **one bag**. They pay **once**, by UPI or card through **Razorpay** or in cash at a central desk, and get **one GST invoice**. The **cash desk** app shows the cashier the change to return and the cash that should be in the drawer at day end. We used **React**, **Node.js + Express**, **SQLite** and **Razorpay**, and followed the **iterative and incremental** model in **three iterations**."

Problem	One queue, payment and GST bill *per showroom*; cash change errors; no reliable end-of-day cash total.
Solution	One bag for the whole mall, one payment, one GST invoice; a cash desk that calculates change and reconciles the drawer.
Actors	Customer, Cashier, and Razorpay (external payment system).
Tech stack	React 19 + Vite 8 + Tailwind 4 (frontend) • Node.js 22 + Express 5 (REST API) • SQLite (`node:sqlite`) • Razorpay.
Scanning	Phone camera; browser `BarcodeDetector` or ZXing-C++ in WebAssembly. Linear barcodes (EAN-13, UPC, Code 128), not QR.
SE model	Iterative and incremental (agile): 3 iterations, each ending in a demo and review.
Diagrams	Architecture, SDLC, Use case, Sequence, Activity, Class (6).
Testing	17 automated tests, all passing; real tags tested (Puma polo, Mufti shirt, JPG parfum).
What's special	No new hardware (uses existing EAN-13 tags + customer's phone) • exact GST for a mixed bag, even when the tag MRP already includes GST • the server calculates every price; the phone only sends what was scanned.

2 Demo script (about 5 minutes)

1. In the `flashcart` folder run `npm run dev`; open the address Vite prints (`localhost:5173`, or `5174` if busy). API runs on port 5005. For the phone camera use `npm run dev:https` (camera needs HTTPS).
2. **Start shopping** → sign in with any 10-digit mobile number (starts with 6–9). *"No password, no app install."*
3. **Scan** or *Type barcode*: Puma `4070034130499`, Mufti `8905310522667`, JPG `8435415076944`, Cappuccino `8903003000034`. Show the beep, toast and Undo.
4. **Bag**: items grouped by showroom, GST per rate, CGST/SGST split. Puma shows Rs. 2999, exactly the printed MRP.
5. **Checkout** → **Cash at the cash desk**: a bill with a Code-128 barcode appears.
6. Second tab: **Cashier login** (`cashier@flashcart.in`, password from `backend/.env`) → *Start the day* → find the bill (scan, bill ID or phone) → cash Rs. 20,000 for Rs. 17,898 → change Rs. 2102 $= 4 \times 500 + 100 + 2$ → receipt turns **Paid**.
7. **End of day**: type the counted cash → report of expected vs. counted, over/short.
8. *Optional (internet):* online payment with Razorpay test card `4100 2800 0000 1007`, any future expiry and CVV → *Success*.

If something fails: camera → use *Type barcode*; need tags → open `/#/labels` and scan from another screen; no internet → show the cash path (works offline).

3 Technologies used

Technology	What it does in FlashCart	Why we chose it
React 19	Builds both web apps (customer app, cash desk) from reusable components.	Component-based, fast screen updates.
Vite 8	Development server and production build; HTTPS mode for phones.	Very fast; small bundle (77.8 KB gzip).
Tailwind CSS 4	Styling and responsive phone/desktop layouts.	Consistent design, little custom CSS.
Axios, Lucide	Sends HTTP requests to the API; icons.	Simple and lightweight.
Barcode engine	Browser `BarcodeDetector` (Chrome), else ZXing-C++ compiled to **WebAssembly**.	Every browser, 25–35 ms per frame, offline.
JsBarcode	Draws the bill barcode (Code 128) and product labels (EAN-13).	Simple SVG output.
Node.js 22+	Runs the server.	Same language as the frontend; built-in SQLite and crypto.
Express 5	REST API under `/api` (e.g. `POST /api/cart/:id/add`).	Minimal, well known, easy to test.
SQLite	Stores customers, cashiers, products, bills, cash shifts.	No DB server, one file, ACID transactions.
Razorpay	Online payments: UPI and cards (test mode).	Indian gateway with UPI and a Node SDK.
Node `crypto`	HMAC-SHA256 (payment check, login tokens) and scrypt (passwords).	Built-in, standard algorithms.
node:test, oxlint, Git	Automated tests, code checking, version control.	Built-in / fast / teamwork.

Terms you should be able to define

SPA: page loads once; React swaps screens without reloading.

REST API: resources at URLs; `GET` read, `POST` create/act, `PUT` update, `DELETE` remove; data as **JSON**.

Three-tier: presentation (React) → logic (Express) → data (SQLite).

WebAssembly: compiled code (here C++) running in the browser at near-native speed.

Hashing vs. encryption: hashing is one-way (passwords); encryption is reversible with a key.

HMAC: a hash made with a secret key; proves data came from someone with the key and wasn't changed.

ACID transaction: changes that happen all together or not at all (save bill *and* reduce stock).

Idempotent: doing it twice = doing it once (verifying a payment twice gives the same receipt).

GST / CGST / SGST / HSN: sale within a state splits GST equally into Central and State GST; HSN = product tax code.

p95: 95% of requests were faster than this value.

Requirements

Functional (what it does)

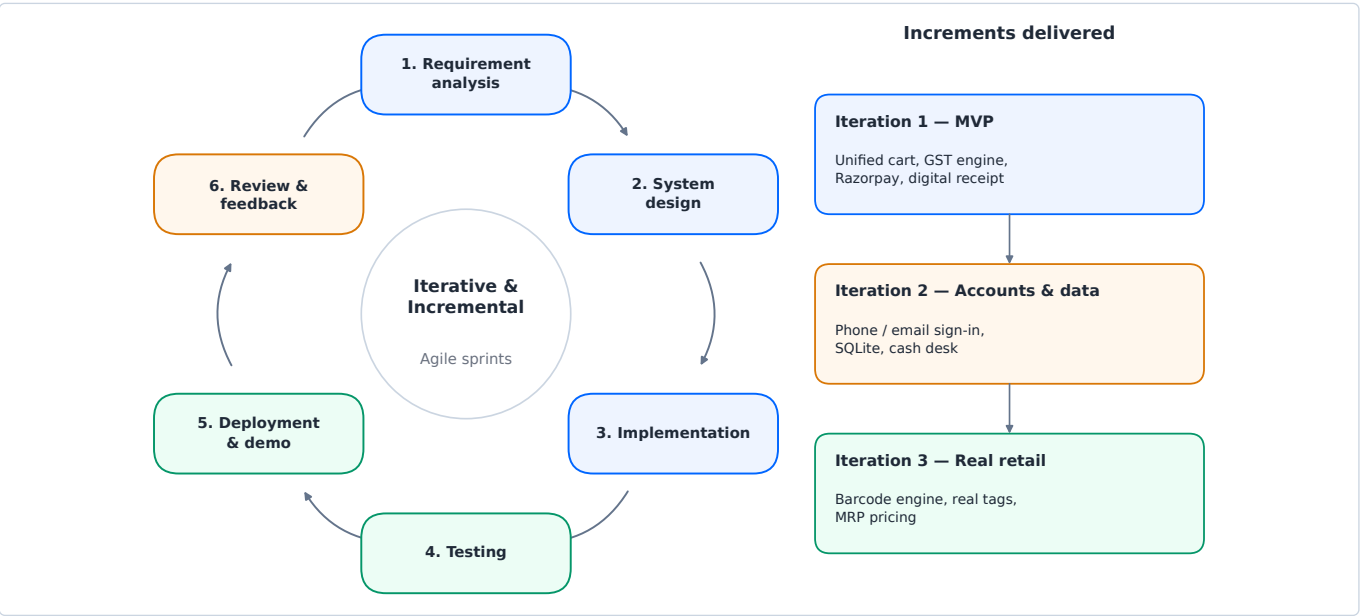
FR1 Customer signs in with a 10-digit mobile number.
FR2 Add products by scanning, typing a barcode or browsing.
FR3 One bag for all showrooms; change quantity, remove.
FR4 GST per rate (5/12/18%), CGST/SGST split, rupee round-off.
FR5 Pay online (Razorpay UPI/card) or choose cash at the desk.
FR6 One GST invoice (digital receipt) per bag.
FR7 Cashier: email + password login, opening cash, find bill, collect cash, see change.
FR8 Close the day: expected vs. counted cash report.

Non-functional (how well)

- **Performance:** add-to-bag < 100 ms (got 0.45 ms); decode < 100 ms (got 25–35 ms).
- **Security:** hashed passwords, signed tokens, role checks, server-side prices, verified payments, login rate limits.
- **Reliability:** data survives restarts; bill + stock saved in one transaction; no duplicate bills.
- **Usability:** any phone browser, no install, manual entry if no camera.
- **Maintainability:** one language; layered code (routes → services → store); 17 tests.

4 Software engineering model: SDLC (report Fig. 2.2)

SAY THIS "We followed the **iterative and incremental model**, the basis of agile. In each of the **three iterations** we did requirement analysis, design, implementation, testing, a demo and a review, and the review feedback decided the next iteration. Each iteration *added* a working increment."



Iterative = we *repeat* the cycle (left circle) and improve each time. **Incremental** = each cycle *adds* working features (right boxes), so there is always a usable product.

The six phases, in FlashCart:

1. **Requirement analysis:** what customers and cashiers need; feedback became requirements.
2. **System design:** architecture, tables, API, UML diagrams.
3. **Implementation:** React apps + Express API for that iteration.
4. **Testing:** automated API tests, browser tests, real tags.
5. **Deployment & demo:** ran on laptop + phone (mid-term demo).
6. **Review & feedback:** what failed or was missing → next iteration.

The three increments:

- **Iteration 1 – MVP (weeks 1–6):** unified cart, GST engine, Razorpay, digital receipt. *Feedback:* data lost on restart; need separate customer/cashier logins; cash desk only for cash; the QR-oriented scanner missed real barcodes.
- **Iteration 2 – Accounts & data (weeks 7–8):** phone login, email login, SQLite, cash desk. *Feedback:* read real retail barcodes, add our own products (MRP includes GST).
- **Iteration 3 – Real retail (weeks 9–10):** new barcode engine, real tags, MRP pricing.

Why not others? **Waterfall** fixes requirements up front and tests at the end; ours changed after every demo and the scanner bug would be found too late. **Spiral** is for large high-risk projects (formal risk analysis). **Pure Scrum** needs formal roles (Product Owner, Scrum Master); we used its short-iteration idea.

Questions ma'am may ask

Q. Which model did you use? Iterative and incremental (agile-style), 3 iterations with all six phases in each.

Q. Example of feedback changing the design? Mid-term demo: scanner failed on real 1D barcodes. Iteration 3 replaced it with the native API / ZXing WebAssembly; all 3 real tags now scan. Real tags print MRP *including* GST, so we added MRP pricing.

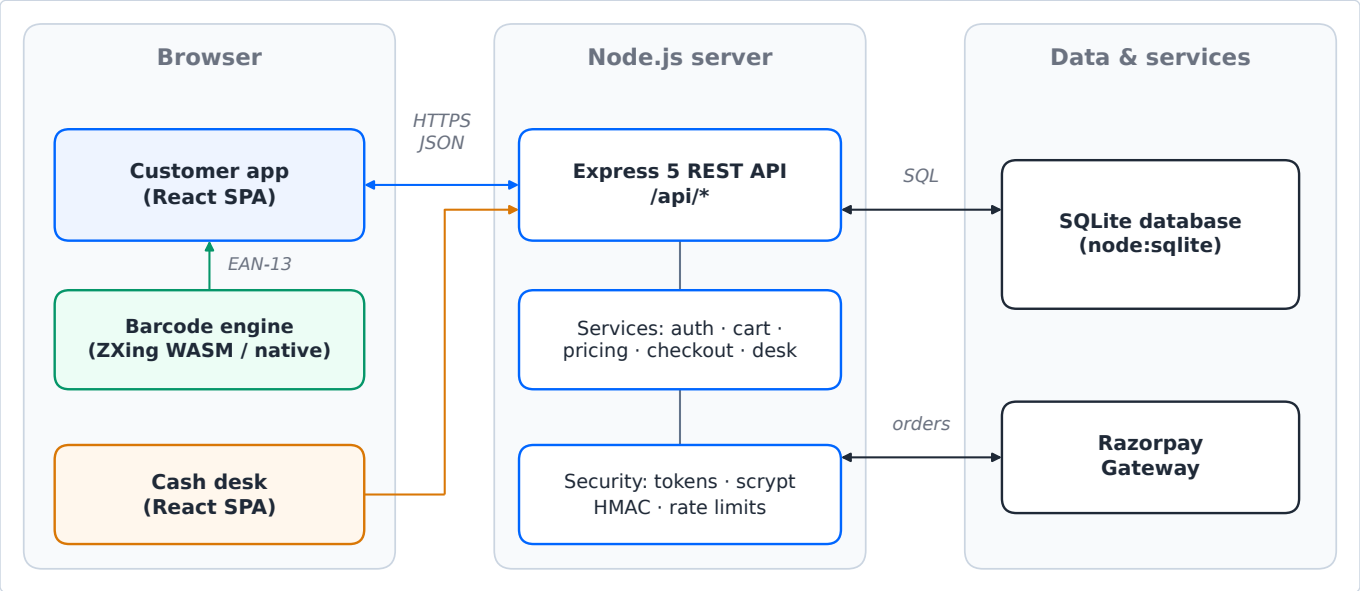
Q. Advantages? Working product early, early testing, requirements can change, lower risk.

Q. Disadvantages? Hard to fix total cost/time up front; needs refactoring and regular feedback.

Q. Is it agile? Yes; iterative-incremental is the core of agile. Each iteration ended in a demo and review.

Q. Which phase made the UML diagrams? System design. The use case diagram also comes from requirement analysis.

5 Architecture diagram (report Fig. 2.1)



The three columns are the three tiers:

1. **Browser (presentation):** two React single-page apps. The *Customer app* (phone) contains the *Barcode engine* (ZXing WASM / native) that turns camera frames into EAN-13 codes. The *Cash desk* runs on the counter laptop or tablet.
2. **Node.js server (application):** one *Express 5 REST API* at `/api/*`; *services* hold the business logic (auth, cart, pricing, checkout, desk); the *security* layer does tokens, scrypt, HMAC and rate limits.
3. **Data & services (data):** the *SQLite database* (via SQL) and the external *Razorpay gateway* (the server creates payment *orders* there).

Arrows: apps ↔ API over **HTTPS + JSON**; engine → app gives the **EAN-13** code; API ↔ DB with **SQL**; API → Razorpay creates **orders**.

One scan, end to end: engine reads `4070034130499` → app sends `POST /api/cart/<bag>/add` with the login token → Express checks the token → cart service finds the product in SQLite and checks stock → pricing rebuilds the bill → JSON back → screen updates (server time < 1 ms).

> **SAY THIS** "Clients send only *intents*, never amounts. The phone says 'add this barcode'; the server looks up price and GST, so a modified app can't change a price."

Main API endpoints

`POST /auth/customer/login`	phone login (token 30 days)
`POST /auth/cashier/login`	email + password (12 hours)
`POST /sessions`	open / resume the bag
`GET /products/:barcode`	barcode lookup
`POST /cart/:id/add`	add item (PUT/DELETE items)
`POST /razorpay/orders`	create payment order
`POST /razorpay/verify`	verify signature, save bill
`POST /checkout/payment`	choose cash (pending bill)
`GET /receipt/:id`	the GST invoice
`POST /pos/lookup`	desk: find by bill ID / phone
`POST /pos/bills/:id/collect`	record cash, get change
`POST /pos/shift/open, /close`	start / close the day

Code layout: `backend/src/` routes, services, store/db, lib, middleware; `backend/test/`; `frontend/src/features/` (landing, customer, pos, receipt, labels) and `lib/`.

Questions ma'am may ask

Q. Why three tiers? Separation of concerns: UI, business rules and data change independently (e.g. SQLite → PostgreSQL without touching the UI).

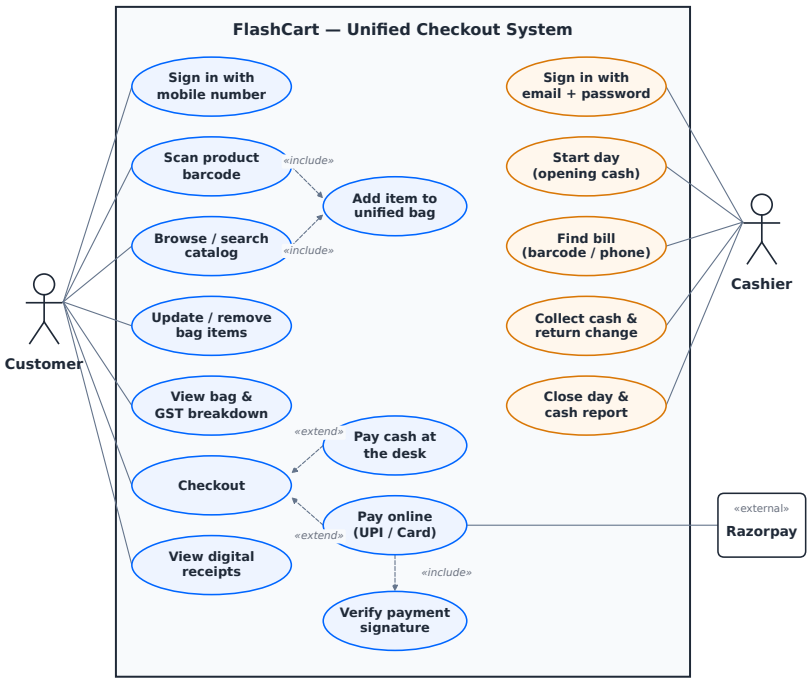
Q. Client–server? Monolith or microservices? Client–server; a *modular monolith* (one server split into services). Microservices would be overkill.

Q. Why carts in memory but bills in SQLite? A bag is short-lived (4-hour expiry) and changes on every scan; a bill is a permanent legal record.

Q. What if the server restarts? Customers, products, bills and shifts are safe in SQLite; only an unpaid open bag is lost.

6 Use case diagram (report Fig. 2.3)

What it is: a behavioural UML diagram showing **what** the system does (functional requirements) and **who** uses it. It does not show order or internal logic.



Symbols

- **Actor:** outside the system, interacts with it (Customer, Cashier).
- «external» **External actor:** another system (Razorpay).
- **System boundary:** inside = our software.
- **Use case:** one goal, verb + noun.
- **Association:** actor takes part.
- «include» **Include:** *always* happens; arrow base → included.
- «extend» **Extend:** *optional / conditional*; arrow extension → base.

Count: 3 actors (Customer, Cashier, Razorpay), 16 use cases (11 customer side, 5 cashier). Colours (blue = customer app, orange = cash desk) are only for readability.

Walk-through

- **Customer** (primary) → 7 use cases: sign in with mobile, scan barcode, browse catalog, update/remove items, view bag & GST, checkout, view receipts.
- **Scan** and **Browse** both **«include» Add item to unified bag**: both end in the same behaviour (check stock, add or qty + 1, re-price), modelled once and reused.
- **Pay cash** and **Pay online «extend» Checkout**: only one happens, by the customer's choice.
- **Pay online «include» Verify payment signature**: every online payment is *always* verified.
- **Razorpay** is linked to Pay online: a secondary actor that supports, never starts.
- **Cashier** (primary) → 5 use cases: sign in (email + password), start day (opening cash), find bill (barcode/phone), collect cash & return change, close day & cash report.

Questions ma'am may ask

Q. Include vs. extend? **Include** is mandatory, arrow base → included (Pay online → Verify signature). **Extend** is optional, runs only under a condition, arrow extension → base (Pay cash → Checkout).

Q. Why is Razorpay an actor? Anything *outside* the boundary that interacts with the system is an actor, including other systems. It is a **secondary** actor.

Q. Primary vs. secondary actor? Primary starts a use case for its goal (Customer, Cashier); secondary helps complete it (Razorpay).

Q. Why isn't the database an actor? It is *inside* our system (implementation), not an external user.

Q. Why isn't sign-in «include»d everywhere? Sign-in is a *precondition*, not a step of each use case; 15 extra arrows would only add clutter.

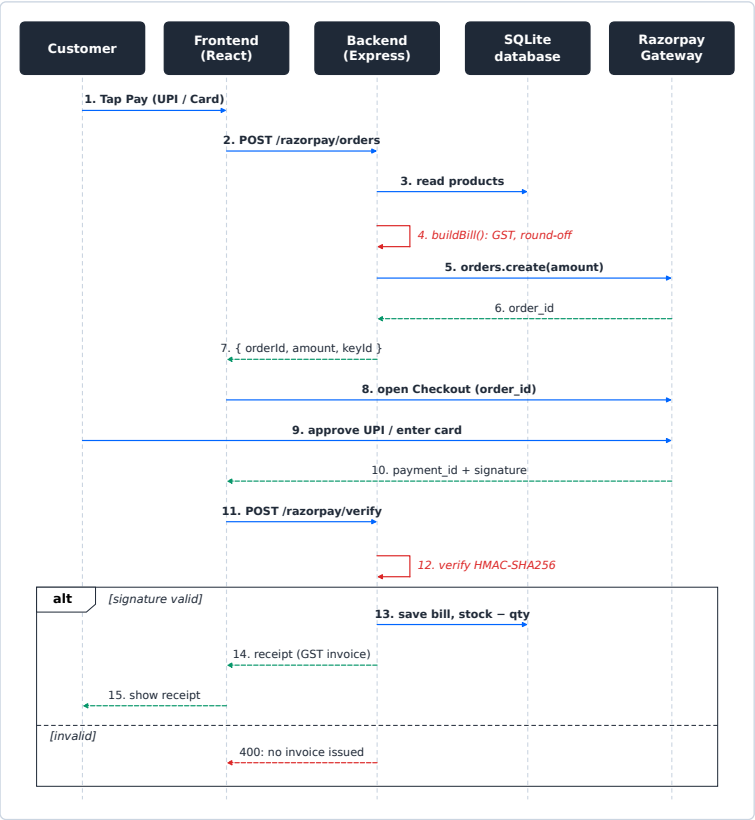
Q. Could you use generalization? Yes: a general *User* actor specialised into Customer and Cashier, or *Pay* specialised into Pay online / Pay cash.

Q. Why no line from Cashier to "Pay cash at the desk"? That is the customer's choice at checkout; the cashier's side is *Find bill* and *Collect cash*.

Q. Describe one use case in text. *Checkout (cash)* – Actor: Customer. Pre: signed in, bag not empty. Flow: choose cash → system re-checks stock, prices bag → pending bill with barcode. Alternate: item out of stock → error. Post: bill waiting at the desk.

7 Sequence diagram (report Fig. 2.4)

What it is: an interaction diagram showing **objects exchanging messages in time order** (top to bottom). It shows **how** one use case runs; ours is *Pay online*.



Step by step

1	Customer taps *Pay* in the React app.
2	Frontend → Backend: `POST /razorpay/orders` with only the bag ID, **no amount**.
3	Backend reads current prices, GST rates, stock from SQLite.
4	*Self-message* `buildBill()`: GST per rate, CGST/SGST, round-off.
5–6	Backend → Razorpay `orders.create(amount)`; gets `order_id`.
7	Backend → Frontend: {orderId, amount, keyId} (public key only).
8–9	Frontend opens Razorpay Checkout; customer pays **directly to Razorpay** (we never see card/UPI details).
10	Razorpay → Frontend: `payment_id` + `signature`.
11	Frontend → Backend: `POST /razorpay/verify`.
12	*Self-message*: `HMAC_SHA256(order_id|payment_id, secret)`, compared in constant time.
13–15	**alt [signature valid]**: save bill + reduce stock in *one* transaction → receipt → shown.
–	**[invalid]**: `400`, no invoice issued.

Symbols: lifeline = participant box + dashed line (its life over time); ——→ **synchronous message** (request, sender waits); ←- - - **return message** (reply); **self-message** = call to itself (4, 12); **alt** fragment = if/else with guards in [ ]. Others: `opt` (if only), `loop`, `par` (parallel). **Activation bar** (thin box while busy) is not drawn in ours.

> **SAY THIS** “The amount is computed only on the server (steps 3–5), card details go straight to Razorpay, and no bill is saved until the signature check in step 12 passes. Only our server knows the secret key, so a fake ‘payment successful’ can’t produce a valid signature.”

Questions ma’am may ask

Q. What is a lifeline? The dashed vertical line under a participant: that object’s existence over time.

Q. Synchronous vs. asynchronous vs. return? Sync (solid, filled arrow): sender waits. Async (solid, open arrow): doesn’t wait. Return (dashed): the reply.

Q. What does `alt` mean? Alternative fragment = if/else; exactly one part runs, chosen by its guard ([valid] / [invalid]).

Q. Why verify on the backend? The secret key must never be in the browser (anyone can read it) and the browser can be tampered with.

Q. Customer closes the Razorpay window? Steps 10–15 never happen; nothing is saved; app shows “Payment cancelled”; they can retry.

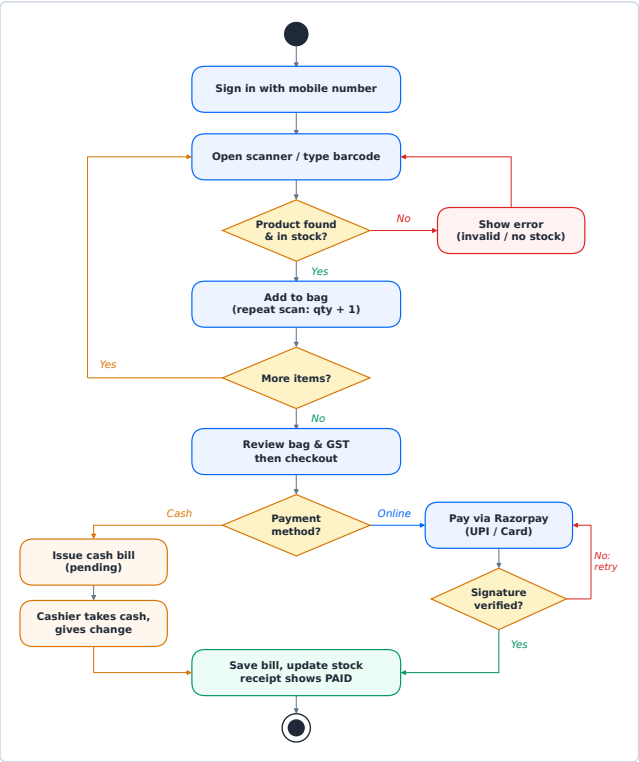
Q. Verify sent twice? Idempotent: the same receipt is returned, never a second bill.

Q. Sequence vs. activity diagram? Sequence: *who* talks to *whom* and when (objects + messages). Activity: flow of steps and decisions in a process.

Q. Communication diagram? Same information as a sequence diagram, organised by object links with numbered messages instead of a time axis.

8 Activity diagram (report Fig. 2.5)

What it is: a behavioural diagram showing the **workflow** of a process: actions, decisions, loops and parallel paths (an advanced flowchart). Ours: the **customer's checkout**, sign-in to paid receipt.



Symbols

- **Initial node:** start.
- **Action:** one step ("Add to bag").
- **Decision:** one in, many out, each with a **guard** [Yes]/[No]. **Merge:** same diamond, many in, one out.
- **Fork / join:** parallel split / wait for all. **Not used in ours.**
- **Final node:** end of the flow.

Walk-through

1. Start → *Sign in with mobile* → *Open scanner / type barcode*.
2. **Product found & in stock?** [No] → *Show error* → back to scanning (**loop 1**). [Yes] → *Add to bag* (re-scan = qty + 1).
3. **More items?** [Yes] → scanner (**loop 2**). [No] → *Review bag & GST, checkout*.
4. **Payment method? Cash:** *Issue cash bill (pending)* → *Cashier takes cash, gives change*. **Online:** *Pay via Razorpay* → **Signature verified?** [No] → retry (**loop 3**), [Yes] → continue.
5. Both paths **merge** at *Save bill, update stock, receipt PAID* → End.

Count: 4 decisions, 3 loops, 2 alternative payment paths, 1 start, 1 end.

> **"FORKS" IN THE REPORT** The report says the flow "forks" on payment method; in UML it is a **decision node**, because only *one* path is taken. A fork (thick bar) would run both in parallel.

> **PRECISE DETAIL** For cash, the bill is saved as **pending** (stock reserved) when issued; collecting the cash marks it **Paid**.

Questions ma'am may ask

Q. Decision vs. fork? Decision picks *one* path by a guard; fork starts *all* paths in parallel.

Q. Merge vs. join? Merge continues when *any* incoming path arrives; join waits for *all* parallel paths.

Q. Where could you use a fork? After payment is verified: "save invoice", "update stock" and "send receipt" in parallel between a fork and a join.

Q. What are guards? Conditions in [ ] on flows leaving a decision: [Yes], [No], [Cash], [Online]; exactly one is true.

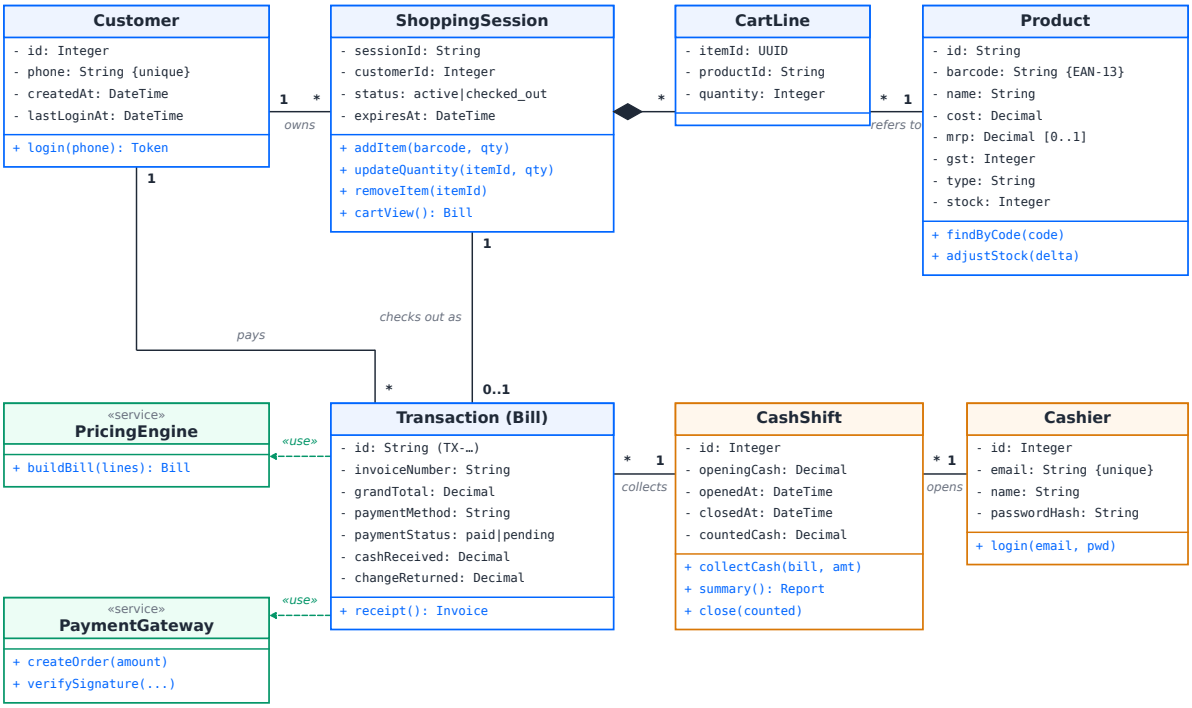
Q. Swimlanes? Why not used? Columns showing *who* does each action. We used colours instead; with lanes: Customer | FlashCart | Razorpay | Cashier.

Q. Activity diagram vs. flowchart? Activity is the UML standard and supports parallel flows, swimlanes and object flows; a flowchart is only sequential.

Q. Activity vs. state diagram? A state diagram shows states of *one* object: Bill *Pending* → *Paid*; Bag *Active* → *Checked out* / *Expired*. Activity shows a whole process.

9 Class diagram (report Fig. 2.6)

What it is: a **structural (static)** diagram of the classes, their attributes and operations, and the relationships between them. Ours is the *domain model*.



Reading a class box: three compartments, *name / attributes / operations*. Visibility − private, + public (private data + public methods = *encapsulation*). Type after a colon (`phone: String`); constraints in braces (`{unique}`, `{EAN-13}`); `mrp [0..1]` = optional (only real tags have an MRP); **«service»** = stereotype for a class with behaviour only.

Relationship symbols

——— **Association**: a link with a name and multiplicities.

◆——— **Composition**: part *cannot exist* without the whole (Session ◆ CartLine).

◇——— **Aggregation** (not used): part can live alone.

- - - -> **Dependency «use»**: uses temporarily (Transaction → services).

———▷ **Generalization** (not used): inheritance.

Classes → code: Customer, Product, Transaction, CashShift, Cashier = DB tables; ShoppingSession + CartLine = in-memory bag (`sessions.js`); PricingEngine = `pricing.js` (`buildBill`); PaymentGateway = `gateway.js` (Razorpay).

Read the relationships aloud

- **Customer 1 – * ShoppingSession** (owns): many bags over time; each bag has one owner.
- **ShoppingSession 1 ◆ * CartLine** (composition): lines die with the bag.
- **CartLine * – 1 Product** (refers to): each line is one product; a product is in many bags.
- **ShoppingSession 1 – 0..1 Transaction** (checks out as): 0 if abandoned.
- **Customer 1 – * Transaction** (pays): many bills over time.
- **Transaction * – 1 CashShift** (collects) and **CashShift * – 1 Cashier** (opens).
- **Transaction - - «use» → PricingEngine, PaymentGateway** (dependency).

TRANSACTION – CASHSHIFT Online bills are never collected at the desk (`shift_id` is NULL), so strictly it should be **0..1**. If ma'am points it out: "Yes ma'am, it should be 0..1."

Questions ma'am may ask

Q. Association vs. aggregation vs. composition? Association: plain link (CartLine–Product). Aggregation (hollow ◇): part can exist alone (department–teachers). Composition (filled ◆): part dies with whole (Session–CartLine; house–rooms).

Q. Why is CartLine composition but Product association? A cart line means nothing outside its bag; a product exists in the catalog anyway.

Q. What is multiplicity? How many objects on each end: `1`, `*` (many), `0..1`, `1..*`.

Q. Why separate «service» classes? Single responsibility: stateless logic, easy to test (GST test calls `buildBill`) and to swap (mock gateway).

Q. Could you use inheritance? Yes: abstract **User** (id, login()) as parent of Customer and Cashier. Kept separate because they log in differently and live in different tables.

Q. Where is Showroom? An attribute (`showroomId`) of Product; could become its own class aggregating Products.

Q. Class vs. object diagram? Class = blueprint (types); object = actual objects at one moment.

Q. Class vs. ER diagram? ER shows only stored data; class diagram adds methods, visibility and non-stored classes (services, bag).

10 Database, core logic, security and testing

SQLite tables

`customers`: id, phone (unique, +91…), created/last login.

`cashiers`: id, email (unique), name, password_hash (scrypt), active.

`products`: id, barcode (EAN-13, unique), sku, name, type, cost, mrp, gst, hsn, showroom_id, stock.

`transactions`: id (TX-…), invoice_number, customer_id, grand_total, method, status, paid_at, shift_id, cash_received, change_returned, `data` (invoice JSON).

`cash_shifts`: id, cashier_id, opened/closed at, opening_cash, counted_cash. `counters`: invoice number per year.

Checks: `gst IN (0,3,5,12,18,28)`, `stock >= 0`. The invoice is stored as a JSON *snapshot* so old bills never change when prices change (deliberate denormalisation).

GST maths (in integer paise, to avoid floating-point errors)

- **Exclusive price:** shirt Rs. 500 + 18% = **Rs. 590**.
- **MRP includes GST:** taxable $=$ MRP$/(1+$ rate$)$. Puma Rs. 2999 at 18% $\rightarrow$ taxable **2541.53** + GST **457.47** (CGST 228.74 + SGST 228.73).
- **Mixed bag:** Puma 2999 (18%) + Earbuds 2000 + 12% = 2240 $\rightarrow$ **Rs. 5239**, one tax line per rate.
- **Round-off:** burger 150 + 5% = 157.50 $\rightarrow$ pay **158** (+0.50).
- **CGST/SGST:** intra-state sale splits GST equally (inter-state would be IGST).
- **Invoice no.** `FC/2627/000003` = FlashCart / FY 2026–27 / running number.

Cash desk: change $=$ received $-$ bill (less is rejected). Greedy note split over 500, 200, 100, 50, 20, 10, 5, 2, 1 (no Rs. 2000: withdrawn by RBI in 2023): $2102 = 4\times500 + 100 + 2$. End of day: expected $=$ opening $+$ cash sales; $5000 + 17898 = 22898$, counted 22,800 $\Rightarrow$ **short 98**.

EAN-13: prefix (country of registration) + company + product + **check digit**. 890 = India (Mufti), 40 = Germany (Puma), 84 = Spain (JPG). Check digit of 4070034130499: odd positions $\times1$ + even $\times3 = 22+39 = 61 \Rightarrow (10-1) =$ **9**. Scanner: rear camera 1920$\times$1080, middle band every 90 ms + full frame every 4th pass, same code ignored for 2.2 s, beep + vibrate + Undo.

Security

- **Signed tokens:** `payload.signature` (HMAC-SHA256 with a secret in `.env`); payload = user, role, expiry. Customers 30 days, cashiers 12 h. Same idea as JWT.
- **Passwords:** scrypt + random salt (N=16384, r=8, p=1): slow and memory-hard; never stored in plain text.
- **Role guards:** customer token at the desk API $\rightarrow$ `403 FORBIDDEN`, and the reverse.
- **Rate limits:** 20 customer / 8 cashier logins per 10 minutes.
- **Server-side pricing**; **payment HMAC** compared in constant time (`timingSafeEqual`).
- **Atomic writes** (bill + stock together); secrets in `.env`, which is git-ignored.

PHONE-ONLY LOGIN “Can’t anyone type my number?” Yes, today. It was a requirement for a quick demo; the fix (**SMS OTP**) is in future scope. Admit it openly.

Testing: 17 automated tests in 7 groups (`npm test`, in-memory SQLite): pricing (1), database (2), auth (4), catalog & cart (4), Razorpay checkout (2), cash desk (2), simulated gateway (2). Types: **unit** (GST engine), **integration** (API + DB), **performance** (90 customers $\times$ 3 scans, decode timing), **user/acceptance** (browser, phone, real tags). Mostly black-box, with white-box edge cases (forged signatures).

Results:

Metric	Target	Achieved
Add-to-bag API (p95)	$<$ 100 ms	0.45 ms
Cash collection API (p95)	$<$ 100 ms	1.13 ms
Barcode decode / frame	$<$ 100 ms	25–35 ms
Real tags decoded	3 / 3	3 / 3
Tests passing	100%	17 / 17
JS download (gzip)	$<$ 150 KB	77.8 KB

Future scope: (1) **inventory** – stock in, low-stock alerts, expiry, admin panel, analytics; (2) **hardware smart cart** – trolley with scanner/RFID, Raspberry Pi/ESP32, touchscreen and a load cell (HX711) that checks weight against scanned items; (3) SMS OTP, Razorpay webhooks, refunds; (4) PostgreSQL, cloud, mall pilot.

11 More questions and cheat sheet

Q. What problem does it solve? Multiple queues, payments and GST bills across showrooms; cash change errors; no reliable day-end total.

Q. Different from a normal POS? One bag and one invoice across *all* showrooms; self-scan on the customer's phone; no new hardware.

Q. Why SQLite, not MySQL/MongoDB? No server to install, one file, ACID, built into Node. Data is relational (customers, bills, shifts). For scale: PostgreSQL.

Q. Why Node.js + Express? One language end to end, built-in DB and crypto, minimal and easy to test.

Q. Why not QR codes? Tags already carry EAN-13; QR would need new labels. QR is disabled so the QR on the same tag isn't read by mistake.

Q. Why HTTPS for the phone camera? Browsers allow the camera only on secure origins (HTTPS or localhost).

Q. Why integer paise? Decimals are approximate $(0.1 + 0.2 \neq 0.3)$; whole paise keep every rupee exact.

Q. MVC? Similar, layered: React = view, Express routes = controllers, services = logic, store/DB = model.

Q. GST changed in Sept 2025 ("GST 2.0"). Most goods are now 5% or 18% (40% luxury). Our 12% is from the spec; the rate is a *column*, so it's a data change, not code.

Q. Jewellery GST is 3%, not 18%. Correct; demo data used 18%. The DB already allows 3%, one value to change.

Q. Two customers buy the last item together? Stock is re-checked at checkout and saved in one transaction; the second gets "out of stock".

Q. Customer leaves without paying? Outside pure software; the smart cart's weight check (future scope) and exit checks of the paid receipt address it.

Q. Internet down? Scanning and cash work on the local network; online payment needs Razorpay.

Q. Hardest part? Reliable 1D barcode reading from a phone camera, and exact GST with tax-inclusive MRP.

Numbers to remember

Tests	17 / 17
Iterations	3 (wk 1–6, 7–8, 9–10)
Diagrams	6
Actors / use cases	3 / 16
Tables	5 + counters
Showrooms / products	5 / 15
GST rates	5, 12, 18%
Token life	30 days / 12 h
Bag expiry / max per line	4 h / 25
Scan interval / decode	90 ms / 25–35 ms
Add-to-bag (p95)	0.45 ms
JS download	77.8 KB gzip

UML notation

«include»	**Include**	always; base → included
«extend»	**Extend**	optional; extension → base
	Sync / return msg	request / dashed reply
	Decision / merge	one path / rejoin
	Fork / join	parallel / wait for all
	Composition	part dies with whole
	Aggregation	part can live alone
	Generalization	inheritance (is-a)
	Dependency	uses temporarily

CLOSING LINE "To sum up: FlashCart turns a whole mall into one checkout. Scan anything, pay once, get one GST invoice, and the cashier always knows the change and the drawer total. Thank you, ma'am."